



UITS
UNIVERSITY OF INFORMATION
TECHNOLOGY AND SCIENCES

University of Information Technology & Sciences

Department of CSE

Software Requirements Specification

Client Portal OS

PortalHQ

“One Portal. Every Client. Every Operation.”

Course Code: CSE0613316

Course Title: Software Project Design and Development Lab

Submission Date: 12 August 2026

Submitted to:

Engineer Md. Safaet Hossain

Designation: Associate Professor & Director

Submitted by:

- | | |
|----------------------------------|----------------|
| 1. MD. Zul kijlee Rahman Tapader | 04324205101002 |
| 2. Khadijatul Kubra | 04324205101006 |
| 3. Mohammad Mutakabbir Hossain | 04324205101007 |
| 4. Badhon Kumar Roy | 04324205101042 |

PortalHQ

“One Portal. Every Client. Every Operation.”

Software Requirements Specification (Version 1.1)

Document Control	
Document Title:	Client Portal OS
Version:	1.1
Prepared by:	Alpha-Numeric
Date:	14 August 2026

REVISION HISTORY			
Date	Version	Description	Author
12 August 2026	1.0	Initial SRS Draft	Alpha-Numeric
14 August 2026	1.1	Added Comparison	Alpha-Numeric

TABLE OF CONTENTS

1	Introduction
1.1	Purpose
1.2	Document Scope & Overview
1.3	Definitions, Acronyms and Abbreviations
1.4	References
2	Product Overview
2.1	Product Vision
2.2	Problem Statement
2.3	Software Comparison
2.4	Objectives
2.4.1	Primary Objectives
2.4.2	Secondary Objectives
2.5	Target Users and User Classes
2.5.1	Platform Administrator
2.5.2	Organization Administrator
2.5.3	Staff
2.5.4	Client
2.5.5	Inventory
2.6	Operating Environment
2.7	Assumptions and Dependencies
2.8	Constraints
3	Scope
4	System Architecture
5	Functional Requirements
5.1	Authentication
5.2	Organization Management
5.3	User Management
5.4	Client Management
5.5	Client Portal
5.6	Project Management
5.7	Milestone Management
5.8	Task or Progress Management
5.9	Document Management
5.10	File Sharing
5.11	Approval
5.12	Support Ticket Management
5.13	Meeting
5.14	Announcement
5.15	Notification
5.16	Asset Management
5.17	Inventory Management
5.18	Inventory Movement

- 5.19 Asset Assignment
- 5.20 Audit logging
- 5.21 Dashboard
- 5.22 Search
- 5.23 Reporting
- 6 External Interface Requirements**
 - 6.1 User Interfaces
 - 6.2 Software Interfaces
 - 6.3 Communication Interfaces
- 7 Non-Functional Requirements**
 - 7.1 Security
 - 7.2 Performance
 - 7.3 Availability
 - 7.4 Scalability
 - 7.5 Usability
 - 7.6 Maintainability
 - 7.7 Reliability
 - 7.8 Portability
 - 7.9 Compliance and Data Privacy
 - 7.10 Localization
- 8 Role-Based Access Control**
- 9 Use Case Description**
 - 9.1 Organization Create Clients
 - 9.2 Client Views Project
 - 9.3 Client Approves Deliverable
 - 9.4 Client Creates Support Ticket
 - 9.5 Inventory Transfer
 - 9.6 Staff Updates Task Progress
 - 9.7 Administrator Generates Report
- 10 Data Model**
 - 10.1 Core Database Entities
 - 10.2 Key Relationships
- 11 Requirements Traceability Matrix**
- 12 Recommended Technology Stack**
- 13 Testing Strategy**
- 14 Security Considerations**
- 15 Risk Management**

APPENDIX

Appendix A: Business Model

Appendix B: Market Positioning & Competitive Differentiation

Appendix C: Expected Benefits

Appendix D: Future Scope

Appendix E: Academic / Software Engineering Value

Appendix F: Success Criteria & Final Project Definition

1. Introduction

1.1 Purpose

PortalHQ is a web-based B2B client portal and business collaboration platform designed for Small and Medium-Sized Enterprises (SMEs). The platform provides businesses with a centralized digital workspace to manage client relationships, projects, documents, communication, approvals, support requests, business assets, inventory movements, and operational activities.

Many SMEs rely on a combination of email, messaging applications, spreadsheets, cloud storage, paper records, and disconnected software systems. This creates information fragmentation, communication gaps, poor accountability, and difficulty tracking business activities. PortalHQ aims to provide a unified platform where a business and its clients can securely collaborate and monitor relevant business activities from a single system.

This document defines the functional and non-functional requirements of the PortalHQ MVP and serves as the primary reference for design, development, testing, and evaluation of the project.

1.2 Document Scope & Overview

Section 2 presents the product overview, including vision, problem statement, objectives, target users, operating environment, assumptions, and constraints. Section 3 defines the MVP scope. Section 4 outlines the system architecture. Section 5 lists functional requirements. Section 6 defines external interface requirements. Section 7 defines non-functional requirements. Sections 8–11 cover access control, use cases, the data model, and requirement traceability. Sections 12–16 describe the technology stack, development strategy, testing strategy, security considerations, and risk management. Appendices A–G provide supporting business context (business model, market positioning, benefits, future scope, academic value, success criteria, and final project definition).

1.3 Definitions, Acronyms and Abbreviations

Term	Meaning
SME	Small and Medium-Sized Enterprise
MVP	Minimum Viable Product
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case
RBAC	Role-Based Access Control

Term	Meaning
JWT	JSON Web Token
API	Application Programming Interface
REST	Representational State Transfer
ERD	Entity Relationship Diagram
UML	Unified Modeling Language
SaaS	Software as a Service
SSO	Single Sign-On
2FA	Two-Factor Authentication
CRUD	Create, Read, Update, Delete
HTTPS	Hypertext Transfer Protocol Secure

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Internal team planning notes and requirement discussions, 2026.
- Course guidelines for the Software Project Design and Development Lab.

2. Product Overview

2.1 Product Vision

PortalHQ aims to become a unified B2B digital workspace where SMEs can manage clients, collaborate on business activities, exchange documents, track projects, handle support requests, and manage business assets from one centralized platform. The platform is designed to be industry-independent.

Potential users include:

- Software companies
- IT service providers
- Construction companies
- Marketing agencies
- Design agencies
- Engineering firms
- Consulting firms

- Event management companies
- Equipment suppliers
- Maintenance service providers
- Professional service businesses
- Other SMEs that regularly work with external clients

2.2 Problem Statement

Many SMEs manage client-facing operations through disconnected tools. For example:

- Client communication → WhatsApp / Messenger / Email
- Documents → Google Drive / OneDrive
- Project progress → Excel / Trello / informal communication
- Approvals → Email / messages
- Support → Phone calls / WhatsApp
- Inventory → Excel / paper records
- Asset movement → Manual registers

As a result:

- Information becomes fragmented.
- Employees spend excessive time searching for information.
- Clients repeatedly request updates.
- Approvals are delayed.
- Documents become difficult to track.
- Business assets are difficult to monitor.
- Inventory movement may not have a proper audit trail.
- Management lacks a centralized operational view.
- Accountability becomes difficult to establish.

PortalHQ addresses these problems by providing a centralized and role-based B2B workspace.

2.3 Software Comparison

Capability	SuiteDash	Clinked	Moxo	PortalHQ
Client Portal	✓	✓	✓	✓
Project Visibility	✓	✓	✓	✓
File Management	✓	✓	✓	✓
Communication	✓	✓	✓	✓
Approval Workflow	✓	✓	✓	✓
Support Tickets	✓/partial	✓	✓	✓
Asset Registry	⚠	⚠	⚠	✓
Inventory	⚠	⚠	⚠	✓
Inventory Movement	✗/limited	✗/limited	✗/limited	✓
Asset Assignment	⚠	⚠	⚠	✓
SME-focused Operations	⚠	⚠	⚠	🎯
Industry-independent	✓	✓	✓	✓

2.4 Objectives

2.4.1 Primary Objectives

1. Centralize SME-client communication.
2. Provide clients with a dedicated digital portal.
3. Improve project and service visibility.
4. Centralize business documents and deliverables.
5. Provide structured approval workflows.
6. Manage support requests through tickets.
7. Track assets and inventory movements.
8. Maintain an auditable activity history.

9. Reduce dependency on fragmented communication channels.
10. Provide SMEs with an organized operational dashboard.

2.4.2 Secondary Objectives

- Improve client transparency.
- Reduce administrative workload.
- Improve accountability.
- Reduce information loss.
- Provide a scalable SaaS foundation.
- Support multiple business types rather than a single industry.

2.5 Target Users & User Classes

2.5.1 Platform Administrator

Responsible for managing the PortalHQ platform. Capabilities:

- Manage organizations
- Manage subscriptions
- Monitor platform activity
- Manage system configuration
- Handle platform-level issues

2.5.2 Organization Administrator

The business owner or administrator. Capabilities:

- Manage organization
- Add employees
- Add clients
- Create projects
- Manage inventory
- Manage assets
- Configure permissions
- Monitor reports
- View audit logs

2.5.3 Staff

Employees working inside the organization. Capabilities depend on assigned permissions, for example:

- Manage projects
- Upload documents
- Respond to tickets
- Manage inventory
- Update project progress

2.5.4 Client

External customer of the organization. Clients should only access information explicitly shared with them. Capabilities may include:

- View projects
- View milestones
- Download documents
- Upload files
- Submit support tickets
- Provide feedback
- Approve deliverables
- View announcements
- View relevant asset information

2.5.5 Inventory

Responsibilities:

- Register assets
- Manage inventory
- Issue items
- Receive items
- Transfer items
- Process returns
- Track asset status
- Maintain movement history

2.6 Operating Environment

- Client-side: any modern desktop or mobile web browser (Chrome, Firefox, Edge, Safari) with internet access.
- Server-side: cloud-hosted or VM-hosted backend running the REST API and business logic layer.
- Database: PostgreSQL relational database server.
- File storage: object storage service (local storage in development; Amazon S3 or compatible in production).

2.7 Assumptions and Dependencies

- Users have access to a modern web browser and a stable internet connection.
- Organizations provide accurate and up-to-date client and employee information.
- File storage functionality depends on the availability of a third-party object storage provider (e.g., Amazon S3).
- Email-based notifications depend on the availability of a third-party SMTP/email delivery service.

- The development team has, or will acquire, working knowledge of the chosen technology stack (React/Next.js, ASP.NET Core or Spring Boot, PostgreSQL).
- The MVP is developed and evaluated in a single-tenant academic context; multi-tenant SaaS operation is a future extension.

2.8 Constraints

- The MVP must be delivered within a four-month timeline by a two-member development team, which limits the scope to the features listed in Section 3.1.
- Budget constraints require the use of free-tier or open-source tools and services during development.
- The system must handle sensitive business and client data and therefore must follow baseline data-protection practices even within the academic MVP (see Section 7 and Section 15).
- Development must follow the technology stack agreed upon by the team (Section 12), as changing the stack mid-project is not feasible within the timeline.

3. Scope

The MVP will include:

- Authentication
- User management
- Organization management
- Client management
- Role-based access control
- Client portal
- Project management
- Milestone management
- Task/progress tracking
- Document management
- File sharing
- Announcements
- Support tickets
- Meeting records
- Approval workflow
- Notifications
- Asset management
- Inventory management
- Inventory movement tracking
- Audit logs
- Dashboard
- Basic reports

4. System Architecture

PortalHQ follows a layered architecture separating presentation, API, business logic, and data layers, so that each layer can be developed, tested, and scaled independently.

Frontend → REST API → Service Layer → Repository / Data Layer → Database

5. Functional Requirements

Each feature below is written in the standard "the system shall..." format, grouped by module and tagged with an MVP priority (Must / Should / Could) using MoSCoW prioritization, so requirements can be mapped directly to the development schedule (Section 13) and to test cases (Section 14).

5.1: Authentication

The system shall allow users to:

- Register where permitted.
- Log in.
- Log out.
- Reset passwords.
- Change passwords.
- Maintain authenticated sessions.

The system shall enforce secure authentication mechanisms.

5.2: Organization Management

An organization administrator shall be able to:

- Create organization profile.
- Update organization information.
- Upload organization logo.
- Manage business settings.
- Manage employees.
- Manage clients.

5.3: User Management

Administrators shall be able to:

- Create users.
- Edit users.
- Deactivate users.
- Assign roles.

- Assign permissions.
- View user activity.

5.4: Client Management

The organization shall be able to:

- Add clients.
- Edit client information.
- Deactivate clients.
- Assign clients to projects.
- View client activity.
- View client-related documents.
- View client-related tickets.

5.5: Client Portal

Each client shall have access to a dedicated portal. The portal shall provide:

- Dashboard
- Projects
- Milestones
- Documents
- Tickets
- Announcements
- Approvals
- Meetings
- Relevant asset/inventory information
- Clients shall only see information authorized by the organization.

5.6: Project Management

Authorized users shall be able to:

- Create projects.
- Update projects.
- Assign project members.
- Define start and end dates.
- Define project status.
- Set project priority.
- Associate clients with projects.

Project statuses:

Planned → Active → On Hold → Under Review → Completed → Cancelled

5.7: Milestone Management

The system shall allow users to:

- Create milestones.
- Assign deadlines.
- Update milestone status.
- Track milestone completion.
- Associate documents with milestones.
- Request client approval.

5.8: Task / Progress Management

Authorized users shall be able to:

- Create tasks.
- Assign tasks.
- Set deadlines.
- Set priority.
- Update task status.
- Record progress.

5.9: Document Management

Users shall be able to:

- Upload documents.
- Download authorized documents.
- Organize documents.
- Categorize documents.
- Associate documents with projects or clients.
- Replace/update document versions.

The system shall maintain document metadata.

5.10: File Sharing

Authorized users shall be able to share files with clients. The system shall record:

- Uploader
- Upload time
- File type
- File size
- Associated project
- Access permissions

5.11: Approval

The system shall support approval workflows. Example:

Deliverable Created → Submitted for Review → Client Reviews → Approved / Rejected → If Rejected → Revision

The system shall record:

- Approver

- Decision
- Date/time
- Comments
- Associated item

5.12: Support Ticket

Clients shall be able to submit support requests. Ticket information shall include:

- Ticket ID
- Subject
- Description
- Category
- Priority
- Status
- Assigned employee
- Created date
- Last updated date

Statuses:

Open → Assigned → In Progress → Waiting for Client → Resolved → Closed

5.13: Meeting Notes

Authorized users shall be able to record:

- Meeting title
- Date
- Participants
- Discussion notes
- Follow-up date

5.14: Announcement Management

Organization administrators shall be able to publish announcements. Announcements may target:

- All clients
- Specific clients
- Specific projects
- Internal employees

5.15: Notification Management

The system shall generate notifications for important events, for example:

- New ticket
- Ticket update
- New document
- Approval request
- Approval result

- Milestone update
- Project announcement
- Inventory transfer
- Asset assignment

5.16: Asset Management

The system shall allow organizations to register physical or digital assets, for example:

- Laptop
- Desktop
- Server
- Printer
- Router
- Project equipment
- Mobile device
- Software license

Each asset shall contain:

- Asset ID
- Asset name
- Category
- Serial number
- Purchase information
- Current status
- Current location
- Assigned user/client
- Warranty information

5.17: Inventory Management

PortalHQ shall provide basic inventory management functionality. The organization shall be able to:

- Create inventory items.
- Define item categories.
- Record stock quantity.
- Record minimum stock level.
- Record suppliers.
- Increase stock.
- Decrease stock.
- View current stock.
- View stock history.

5.18: Inventory Movement

The system shall maintain a complete movement history. Supported movements:

Receive → Store → Issue → Transfer → Return → Maintenance → Dispose

Each movement shall record:

- Movement ID
- Item/Asset
- Quantity
- From location
- To location
- Responsible user
- Date/time
- Reason
- Reference
- Notes

5.19: Asset Assignment

Authorized users shall be able to assign assets to:

- Employees
- Departments
- Projects
- Clients
- Locations

The system shall maintain assignment history.

5.20: Audit Logging

The system shall record important system activities, for example:

- Login
- Logout
- File upload
- File deletion
- Project creation
- Project modification
- Approval
- Ticket update
- Inventory movement
- Asset assignment
- User permission change

Audit records shall include:

- User
- Action
- Object
- Timestamp
- Relevant metadata

5.21: Dashboard

The organization dashboard shall provide summarized information, for example:

- Active projects
- Completed projects
- Open tickets
- Pending approvals
- Total assets
- Available assets
- Inventory quantity
- Recent movements
- Recent activity

The client dashboard shall provide:

- Active projects
- Milestone progress
- Pending approvals
- Recent documents
- Open tickets
- Recent announcements

5.22: Search

Users shall be able to search authorized information. Searchable entities may include:

- Projects
- Clients
- Documents
- Tickets
- Assets
- Inventory items
- Users

5.23: Reporting

The system shall generate basic reports, for example:

- Project progress report
- Ticket report
- Asset report
- Inventory report
- Inventory movement report
- Client activity report

6. External Interface Requirements

6.1 User Interfaces

- A responsive, browser-based web application built with a component-based frontend framework (React.js).
- Distinct interface views for Platform Administrator, Organization Administrator, Staff, and Client, reflecting each role's permissions.
- A dashboard-centric layout with a persistent navigation panel (Organization, Clients, Projects, Tickets, Assets, Inventory, Reports).
- A dedicated, simplified Client Portal view limited to information explicitly shared with that client.

6.2 Software Interfaces

- MySQL relational database for persistent storage of all business data.
- A RESTful API layer connecting the frontend to backend business logic.
- Object storage service for documents and files.

6.3 Communication Interfaces

All client–server communication shall use HTTPS to encrypt data in transit. The REST API shall exchange data using JSON payloads over HTTP(S).

7. Non-Functional Requirements

7.1: Security

The system shall:

- Use encrypted communication through HTTPS.
- Hash passwords securely.
- Implement RBAC.
- Validate user permissions.
- Protect unauthorized resources.
- Maintain audit logs.
- Implement secure file access.

7.2: Performance

For normal operations, the system should provide acceptable response time under the expected MVP user load. Target: typical API response time ≤ 2 seconds under normal conditions.

7.3: Availability

The system should be designed for high availability during normal business operations.

7.4: Scalability

The architecture should allow future expansion to:

- Multiple organizations
- Thousands of users
- Larger document repositories
- Additional modules
- Mobile applications
- Third-party integrations

7.5: Usability

The interface should:

- Be responsive.
- Work on desktop and mobile browsers.
- Use consistent navigation.
- Provide clear feedback.
- Minimize unnecessary steps.

7.6: Maintainability

The system shall use:

- Modular architecture
- RESTful APIs
- Clear separation of concerns
- Version-controlled source code
- Coding standards
- Documentation
- Automated testing where applicable

7.7: Reliability

The system shall maintain data consistency during:

- Inventory transactions
- Asset transfers
- Approvals
- Ticket updates
- File operations

7.8: Portability

The system shall function correctly across modern browsers (Chrome, Firefox, Edge, Safari) without requiring browser-specific code, and the backend shall be deployable on any standard cloud or virtual-machine environment without architectural changes.

7.9: Compliance & Data Privacy

The system shall handle organization, client, and personal data according to baseline data-protection practices — data minimization, role-based access control, and secure storage — and shall be architected so that stronger regulatory compliance (e.g., GDPR-aligned data-subject rights) can be added as the platform scales beyond the academic MVP.

7.10: Localization

The MVP shall support the English language only. The architecture shall not prevent future addition of multiple languages, currencies, and regional date/number formats.

8. Role-Based Access Control

Example permission model (the exact permission model will be finalized during system design):

Feature	Admin	Staff	Client	Inventory Manager
Organization	Full Access	—	—	—
Users	Full Access	Limited	—	—
Clients	Full Access	Limited	Own Profile	—
Projects	Full Access	Limited	View	—
Documents	Full Access	Limited	Authorized	—
Tickets	Full Access	Limited	Own Tickets	—
Approvals	Full Access	Limited	Own Requests	—
Inventory	Full Access	Limited	—	Full Access
Asset Movement	Full Access	Limited	Limited	Full Access
Reports	Full Access	Limited	Limited	Full Access
Audit Logs	Full Access	Limited	—	Limited

9. Use Case Descriptions

The core use cases below are written in a structured format (actor, precondition, main flow, alternate flow, postcondition) so they can be directly converted into UML use-case diagrams and test scenarios during design.

91: Organization Creates Client

Field	Description
Actor(s)	Organization Administrator
Precondition	Administrator is logged in with valid permissions.
Main Flow	- Administrator navigates to Client Management. - Administrator selects "Add Client". - Administrator enters client information (name, contact details, etc.). - Administrator saves the record.

	5. System creates the client account and sends access credentials/invitation.
Alternate Flow	If required fields are missing or invalid, the system displays a validation error and the client is not created.
Postcondition	A new client account exists and is linked to the organization.

9.2: Client Views Project

Field	Description
Actor(s)	Client
Precondition	Client has valid login credentials and at least one assigned project.
Main Flow	1. Client logs into the portal. 2. System displays the client dashboard. 3. Client navigates to "Projects". 4. Client selects a project. 5. System displays project progress and milestones.
Alternate Flow	If no projects are assigned, the system displays an empty-state message.
Postcondition	Client has viewed up-to-date project progress information.

9.3: Client Approves Deliverable

Field	Description
Actor(s)	Employee, Client
Precondition	A deliverable has been uploaded and is ready for review.
Main Flow	1. Employee uploads the deliverable and submits it for approval. 2. System notifies the client. 3. Client reviews the deliverable. 4. Client approves or rejects the deliverable, optionally adding comments.

	5. System records the decision.
Alternate Flow	If rejected, the system flags the deliverable for revision and notifies the employee.
Postcondition	The deliverable's approval status is recorded and visible to both parties.

94: Client Creates Support Ticket

Field	Description
Actor(s)	Client, Staff
Precondition	Client is logged into the portal.
Main Flow	1. Client navigates to Support. 2. Client selects "Create Ticket". 3. Client enters subject, description, category, and priority. 4. Client submits the ticket. 5. System notifies assigned staff. 6. Staff resolves the issue and updates the ticket status. 7. Client confirms resolution. 8. System closes the ticket.
Alternate Flow	If the client is unsatisfied with the resolution, the ticket can be reopened for further review.
Postcondition	The support ticket is resolved and closed with a complete activity history.

9.5: Inventory Transfer

Field	Description
Actor(s)	Inventory / Asset Manager
Precondition	The inventory item exists in the system with available stock.
Main Flow	1. Inventory Manager selects the item to transfer. 2. Inventory Manager selects "Transfer".

	3. Inventory Manager specifies source and destination locations. 4. Inventory Manager enters quantity and reason. 5. Inventory Manager confirms the transfer. 6. System updates inventory quantities and records the movement.
Alternate Flow	If the requested quantity exceeds available stock, the system displays an error and blocks the transfer.
Postcondition	Inventory levels are updated and the transfer is logged in the movement history.

9.6: Staff Updates Task Progress

Field	Description
Actor(s)	Staff / Employee
Precondition	Staff member is assigned to a task within an active project.
Main Flow	1. Staff logs in and navigates to assigned tasks. 2. Staff selects a task. 3. Staff updates task status and progress notes. 4. System saves the update and notifies relevant stakeholders.
Alternate Flow	If the task is marked complete, the system prompts for confirmation before finalizing.
Postcondition	Task status and overall project progress are updated in real time.

9.7: Administrator Generates Report

Field	Description
Actor(s)	Organization Administrator
Precondition	Administrator has access to the reporting module and relevant data exists.
Main Flow	1. Administrator navigates to Reports. 2. Administrator selects a report type (e.g., project progress, inventory).

	3. Administrator applies filters (date range, project, client). 4. System generates and displays the report. 5. Administrator exports the report if needed.
Alternate Flow	If no data matches the filters, the system displays an empty report with a notice.
Postcondition	Administrator obtains an accurate operational report to support decision-making.

10. Data Model

10.1 Core Database Entities

Initial database design may contain the following entities (relationships will be finalized during ERD development):

Entity Group	Entities
Identity & Access	User, Role, Permission
Organization	Organization, Client
Project Delivery	Project, ProjectMember, Milestone, Task
Documents	Document, File
Approvals & Support	Approval, Ticket, TicketComment, Meeting
Communication	Announcement, Notification
Assets & Inventory	Asset, AssetAssignment, InventoryItem, InventoryTransaction, Location, Supplier
Governance	AuditLog

10.2 Key Relationships

- One Organization has many Users, Clients, Projects, Assets, and Inventory Items.
- One Project belongs to one Organization and one Client, and has many Milestones and Tasks.
- One Milestone can have many associated Documents and may require a Client Approval.
- One Ticket belongs to one Client and is assigned to one Staff member; a Ticket has many TicketComments.
- One Asset can have many AssetAssignment records, forming its assignment history.

- One InventoryItem has many InventoryTransaction records, forming its movement history.
- One User has one Role, and one Role has many Permissions.
- One Organization has many AuditLog entries recording activity across all modules.

11. Requirements Traceability Matrix

This simplified matrix maps key functional requirements to the use cases and modules that realize them, to support verification during testing.

Functional Requirement(s)	Related Use Case / Module
FR-01 Authentication	Login / session module (all use cases depend on it)
FR-02, FR-03 Organization & User Management	Admin back-office module
FR-04, FR-05 Client Management & Client Portal	UC-01, UC-02
FR-06, FR-07, FR-08 Project, Milestone & Task Management	UC-02, UC-06
FR-09, FR-10 Document Management & File Sharing	UC-03
FR-11 Approval Management	UC-03
FR-12 Support Ticket Management	UC-04
FR-13, FR-14, FR-15 Meetings, Announcements & Notifications	Communication module
FR-16, FR-17, FR-18, FR-19 Assets & Inventory	UC-05
FR-20 Audit Logging	Cross-cutting — logs actions from all use cases
FR-21, FR-22, FR-23 Dashboard, Search & Reporting	UC-07

12. Recommended Technology Stack

Frontend

- React.js
- Component-based development

- Responsive UI
- Strong ecosystem
- Suitable for dashboard-heavy applications

Backend

Django and Django Rest Framework

Database

MySQL. Reasons:

- Reliable
- Open source
- Strong relational model
- Suitable for transactional systems
- Excellent for complex relationships

File Storage

- Development: local/object storage.

Authentication

- Role-Based Access Control

Version Control

- Git + GitHub

13. Testing Strategy

PortalHQ should use multiple levels of testing.

Unit Testing

Test individual:

- Services
- Controllers
- Business rules
- Utility functions

Integration Testing

Test:

- API + database
- Authentication
- File management
- Inventory transactions
- Approval workflow

System Testing

Test complete workflows. Example:

Client Created → Project Created → Milestone Created → Deliverable Uploaded → Client Approval → Project Completed

Security Testing

Test:

- Unauthorized access
- Broken access control
- Invalid authentication
- File access
- Session management
- Input validation

User Acceptance Testing

Test with representative SME workflows.

14. Security Considerations

Because PortalHQ stores business and client information, security is a core requirement rather than an optional feature. The MVP should implement:

- Password hashing
- HTTPS
- RBAC
- Server-side authorization
- Input validation
- File type validation
- Secure file access
- Audit logs
- Session/token management
- Database constraints
- Protection against common web vulnerabilities

Potential future security improvements:

- Two-factor authentication
- Single Sign-On
- Security event monitoring

15. Risk Management

The following risks have been identified for the four-month MVP timeline, along with mitigation strategies.

Risk	Impact	Likelihood	Mitigation
Small team size (2 developers) causes schedule delay	High	Medium	Strict MoSCoW scope control; weekly sprint reviews; defer Could-priority items first if behind schedule
Feature creep beyond agreed MVP scope	High	Medium	Maintain the Out-of-Scope list; route new ideas to Future Scope

Risk	Impact	Likelihood	Mitigation
Third-party service downtime (file storage / email)	Medium	Low	Use reliable providers; add retry/backoff logic
Security breach involving sensitive B2B/client data	High	Low	Enforce RBAC, HTTPS, password hashing, and audit logging from Month 1
Underestimated time for testing and bug-fixing	Medium	Medium	Reserve all of Month 4 for testing, stabilization, and documentation
Requirement misunderstanding between team and supervisor	Medium	Medium	Regular SRS reviews and sign-off checkpoints with the course supervisor

Appendix A: Business Model

PortalHQ can follow a SaaS model.

Free / Trial

Limited:

- Users
- Clients
- Projects
- Storage

Professional

For small businesses. Possible features:

- More users
- More projects
- Increased storage
- Inventory
- Reports
- Advanced permissions

Business

For growing SMEs. Possible features:

- Multiple departments
- Advanced audit logs
- Custom branding
- Advanced reports

- API access

Enterprise

- Dedicated deployment
- SSO
- Custom integrations
- Advanced security
- Dedicated support

Pricing should be validated through market research rather than fixed during the academic MVP.

Appendix B: Market Positioning & Competitive Differentiation

PortalHQ should not position itself as "another project management system." Instead:

PortalHQ is a B2B client collaboration and operational portal designed for SMEs that need one centralized workspace for client communication, project delivery, documents, approvals, support, and business asset/inventory tracking.

The key differentiator is the combination of Client Portal + Collaboration + Project Visibility + Support + Asset/Inventory Operations within a single SME-oriented platform.

Competitive Landscape

Generic tools usually focus on one or two areas:

Category	Typical Focus
Project Management Tools	Internal task/project management
Cloud Storage	File storage
Messaging Apps	Communication
Helpdesk Tools	Support
Inventory Software	Stock management
CRM	Sales/customer relationships

PortalHQ aims to connect selected client-facing operational workflows into one system. The objective is not to replace every existing business application, but to provide SMEs with a unified operational layer for client collaboration.

Appendix C: Expected Benefits

For SMEs

- Centralized information
- Reduced communication overhead
- Better accountability
- Improved document organization

- Faster approvals
- Better inventory visibility
- Better client service
- Reduced dependency on spreadsheets
- Professional client experience

For Clients

- One place to access information
- Better project visibility
- Faster communication
- Easy document access
- Structured approvals
- Support tracking
- Transparent project progress

Appendix D: Future Scope

PortalHQ can evolve into a broader SME business operating platform. Potential future modules:

Client Relationship

- CRM
- Client onboarding
- Client feedback
- Communication history

Finance

- Invoicing
- Payment tracking
- Expense tracking

Operations

- Staff assignment
- Resource planning
- Procurement

Inventory

- Purchase orders
- Supplier management
- Stock forecasting
- Barcode/QR support

Communication

- Live chat
- Email integration
- Calendar integration

Security

- 2FA
- SSO

- Advanced audit system

Integrations

- Google Workspace
- Microsoft 365
- Slack
- GitHub
- Jira
- Accounting systems

Platform

- Mobile application
- White-label portal
- Multi-tenant SaaS
- Public API
- Plugin architecture

Appendix E: Academic / Software Engineering Value

PortalHQ provides significant scope for demonstrating software engineering principles. The project will involve:

- Requirement Engineering
- System Analysis
- UML Modeling
- Use Case Modeling
- Database Design
- Software Architecture
- REST API Design
- Authentication
- Authorization
- RBAC
- Workflow Design
- Transaction Management
- File Management
- Testing
- Security Engineering
- Version Control
- Deployment
- Software Documentation

Therefore, the project is not merely a CRUD application. It represents a multi-module B2B software system with real-world business rules and security requirements.

Appendix F: Success Criteria & Final Project Definition

Success Criteria

The MVP will be considered successful if a representative SME can, using a single platform:

1. Create an organization.
2. Add employees and clients.
3. Create a project.
4. Invite a client.
5. Share project information.
6. Upload documents.
7. Create milestones.
8. Request client approval.
9. Receive support tickets.
10. Track assets.
11. Record inventory movements.
12. View operational activity.
13. Generate basic reports.

Final Project Definition

Field	Description
Product Name	PortalHQ
Product Category	B2B SaaS / Client Collaboration & Business Operations Platform
Target Market	Small and Medium-Sized Enterprises (SMEs)
Core Problem	Fragmented client communication and business operational information
Core Solution	A centralized portal connecting SMEs with their clients while organizing projects, documents, approvals, support, assets, and inventory operations
MVP Duration	4 Months
Team	2 Developers
Future Direction	Multi-tenant SaaS → SME Business Operations Platform

One-Line Project Statement

PortalHQ is a secure B2B client collaboration and business operations platform that enables SMEs to manage client interactions, projects, documents, approvals, support, assets, and inventory movements through a centralized digital workspace.

“PortalHQ — One Portal. Every Client. Every Operation.”